

Senior C# Coding Assignment: Highway Speed Enforcement Pipeline

Introduction:

Modern smart-city infrastructure relies heavily on processing massive streams of IoT data in real-time. Whether tracking public transit, toll booths, or speed traps, backend systems must ingest, process, and broadcast high-frequency data with minimal latency.

Your task is to build a prototype for a real-time **Highway Speed Enforcement Pipeline**.

Imagine a stretch of a multi-lane urban highway equipped with a series of **10 sequential speed cameras** placed one after another in a single direction of travel. As vehicles drive down the highway, they trigger these cameras in chronological order (Camera 1, then Camera 2, up to Camera 10).

The system consists of three separate, decoupled applications working in tandem:

1. **Telemetry data generator** (console application)
 - ↓ (emits telemetry data over any communication protocol)
2. **Data processor** (service running inside a Docker container)
 - ↓ (provides REST endpoint)
3. **Client** (desktop application)

We respect your time and don't expect a pixel-perfect solution. Instead, we want to focus on your design choices and how you structure the underlying solution.

Implementation:

1. The Telemetry Data Generator

A standalone console application that acts as the mock data producer.

To simulate traffic flow, the generator application runs a continuous loop that updates once per second and manages a central registry of active vehicles. To ensure a completely predictable state machine, the loop must execute these steps in order every second:

1. **New Vehicle Influx:** Spawn exactly 100 new, unique vehicle number plates and add them to the active registry (Initial Camera Counter = 0, meaning they haven't hit a camera yet).
2. **Sequential Camera Progression & Emission:** Advance the camera counter for every active vehicle in the registry by 1, and immediately emit a telemetry message stating it was captured by Camera # n.
3. **Vehicle Exit (Cleanup):** Identify and drop any vehicle from the registry whose counter has exceeded 10 (meaning it has passed Camera 10 and left the monitored sector).

Technical Specs:

- **Data Payload:** Each emitted message must contain: string NumberPlate, long TimestampTicks, float CurrentSpeed, and int CameraId (from 1 to 10).
- **Speed Dynamics:** Speeds must be handled as float values representing **km/h**. The vehicle's speed must vary naturally and realistically between cameras (e.g., fluctuating around a baseline highway speed limit of 130 km/h).
- **Network Protocol:** The choice of network transport (e.g., gRPC, MQTT, WebSockets, HTTP REST, or raw TCP sockets) from the Generator to the Processor is left entirely up to you.

Simplification Note: In a real-world scenario, the time between camera captures would depend dynamically on the vehicle's speed. For the sake of keeping this assignment focused on architecture rather than physics modeling, we have simplified the timeline so that all vehicles advance by exactly one camera every second. This means each vehicle has a precise lifespan of 10 seconds and emits 10 total messages.

2. The Telemetry Data Processor (Docker Service)

A containerized backend service running in Docker Desktop (.NET 8/9/10) that acts as the ingestion engine, analytics calculator, and data provider.

Ingestion & Metrics Calculation:

The service must ingest the continuous stream of telemetry messages from the Generator, parse them concurrently, and maintain two distinct real-time analytics registries in a thread-safe manner:

1. Per-Camera Leaderboards: The processor must maintain 10 individual registries (one for each camera). Each tracks a strictly bounded Top 10 list of the fastest vehicles based on the instantaneous CurrentSpeed recorded at the moment they passed through that specific camera.
2. Global System Leaderboard (Live Active Traffic Only): The processor must track a single, system-wide Top 10 leaderboard of the vehicles with the highest overall average speed.
 - The Equation: As a vehicle moves through the highway, the system tracks its cumulative average speed across all cameras it has hit so far.
 - The Gate: To prevent early-stage skew (where a single high speed at Camera 1 unfairly pollutes the global leaderboard), a vehicle only qualifies for this leaderboard after it has passed at least 3 cameras.
 - The Purge: This leaderboard represents all recorded traffic. The moment a vehicle passes Camera 10 and it does not score on any leaderboard, its tracking data will be fully purged.

Constraints:

- Thread Safety:
The ingestion engine will receive hundreds of concurrent messages. All lookups, average calculations, and leaderboard updates must use non-blocking, thread-safe collections or fine-grained locking.
- Tie-Breaking:
In the event of an exact speed match on any leaderboard, resolve the tie deterministically by sorting alphabetically by NumberPlate.

REST API:

The processor must expose a lightweight REST API endpoint (GET /api/traffic-snapshot).

The returned JSON payload must be a Data Transfer Object (DTO) containing:

- A collection of the 10 camera leaderboards.
- A collection of the single global average leaderboard.
- A UTC timestamp of when the snapshot was generated.

3. The WPF Desktop Client

A standalone desktop application providing a real-time visual dashboard of the highway's speed enforcement data.

- **Throttled Data Fetching:** The client must pull data from the Docker service's REST API using a background polling loop operating at exactly a 3-second interval. The UI thread must only be invoked to update the visual data-bindings.
- **The Dashboard View:** The interface must cleanly display:
 - A global panel showing the Global Top 10 highest average speed vehicles.
 - A selectable panel or grid showing the Top 10 fastest vehicles for the individual cameras (e.g., allowing a user to select "Camera 5" to see its specific speeding list).